SQL et bases de données relationnelles

Informatique — Licence 2 (L2)

Cours INF-L2-04

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours
Aller au-delà du SELECT de base : ordre d'exécution des requêtes, filtrage fin (LIKE, BETWEEN, IN), jointures internes et externes, agrégations avec GROUP BY et HAVING, sous-requêtes, création de tables (DDL), vues et index pour optimiser.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Le langage SQL : structure et ordre d'exécution des requêtes
• Leçon 2 — Filtrage avancé : WHERE, LIKE, BETWEEN et IN
• Leçon 3 — Jointures internes et externes
• Leçon 4 — Fonctions d'agrégation et regroupements GROUP BY et HAVING
• Leçon 5 — Sous-requêtes et requêtes imbriquées
• Leçon 6 — Création et modification de tables : le DDL
• Leçon 7 — Vues, index et optimisation simple des requêtes
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

En première année, tu as appris à interroger une base de données : SELECT, FROM, un premier WHERE, un premier ORDER BY. Ce cours de L2 part de là et change d'échelle : les vraies questions métier — « quel est mon chiffre d'affaires par catégorie ? », « quels clients n'ont jamais commandé ? », « quelles commandes dépassent la commande moyenne ? » — exigent de combiner plusieurs tables, de regrouper les lignes, de calculer des totaux et parfois d'imbriquer une requête dans une autre.

Tout le cours travaille sur une seule base d'exemple, celle d'une petite boutique informatique : 7 clients, 9 produits, 8 commandes et leurs lignes. Ce schéma volontairement petit permet de vérifier chaque résultat à la main. Toutes les requêtes du cours ont été exécutées sous SQLite et les résultats affichés sont ceux réellement obtenus : quand tu refais les exemples sur ta machine, tu dois retrouver exactement les mêmes lignes — sinon, cherche l'écart, c'est là que se cache l'apprentissage.

Le cours se termine par l'autre visage de SQL, souvent négligé : le langage de définition de données (DDL), qui crée et fait évoluer la structure même de la base, puis par deux outils de professionnel, les vues et les index. À la fin, tu sauras non seulement poser des questions complexes à une base, mais aussi organiser la base pour que ces questions restent simples et rapides.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Expliquer l'ordre logique d'exécution d'une requête SELECT et prédire son résultat.
- Filtrer finement des données avec WHERE, LIKE, BETWEEN, IN et les opérateurs logiques.
- Combiner plusieurs tables avec des jointures internes (INNER JOIN) et externes (LEFT JOIN).
- Calculer des statistiques par groupe avec COUNT, SUM, AVG, MIN, MAX, GROUP BY et HAVING.
- Écrire des sous-requêtes scalaires, avec IN et dans la clause FROM pour répondre à des questions en deux temps.
- Créer et modifier la structure d'une base avec le DDL : CREATE TABLE, ALTER TABLE, contraintes.
- Simplifier et accélérer les requêtes grâce aux vues et aux index, et lire un plan d'exécution simple.

Leçon 1 — Le langage SQL : structure et ordre d'exécution des requêtes

SQL (Structured Query Language) est le langage standard des bases de données relationnelles : MySQL, PostgreSQL, SQLite, Oracle ou SQL Server parlent tous ce même langage, avec de petites variations de dialecte. Il se découpe en familles : le DML pour manipuler les données (SELECT, INSERT, UPDATE, DELETE), le DDL pour définir la structure (CREATE, ALTER, DROP — leçon 6), et le DCL pour gérer les droits d'accès. Ce cours se concentre sur le SELECT avancé, puis sur le DDL.

Une requête SELECT complète s'écrit avec six clauses : SELECT (les colonnes), FROM (les tables), WHERE (le filtre sur les lignes), GROUP BY (les regroupements), HAVING (le filtre sur les groupes) et ORDER BY (le tri), auxquelles s'ajoute souvent LIMIT pour restreindre le nombre de lignes rendues. Mais attention : l'ordre dans lequel on ÉCRIT les clauses n'est pas l'ordre dans lequel le moteur les EXÉCUTE. Logiquement, le moteur traite d'abord FROM (d'où viennent les données), puis WHERE (quelles lignes garder), puis GROUP BY, puis HAVING, puis SELECT (quelles colonnes calculer) et enfin ORDER BY et LIMIT.

Comprendre cet ordre logique explique la plupart des erreurs des débutants. Un alias créé dans le SELECT n'existe pas encore quand le WHERE s'exécute : on ne peut donc pas filtrer dans le WHERE sur un alias de colonne calculée. De même, une fonction d'agrégation comme SUM n'a de sens qu'après le regroupement : elle est interdite dans le WHERE, c'est le HAVING qui prend le relais. Retenir l'ordre FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT, c'est détenir la clé de lecture de toutes les requêtes de ce cours.

L'essentiel
SQL interroge (SELECT), manipule (DML), structure (DDL) et protège (DCL). L'ordre d'ÉCRITURE des clauses diffère de leur ordre d'EXÉCUTION logique : FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT.

Exemple — Une requête complète sur la table produits de la boutique

```
-- Base « boutique » : produits(id_produit, nom_produit,
-- categorie, prix, stock)
-- Question : les produits de plus de 20 000 FCFA,
-- du plus cher au moins cher.
SELECT nom_produit, prix
FROM produits
WHERE prix > 20000
ORDER BY prix DESC;

-- Résultat (6 lignes) :
-- Ordinateur portable | 350000
-- Tablette 10 pouces | 120000
-- Imprimante laser | 95000
-- Smartphone X12 | 85000
-- Disque dur externe 1To| 45000
-- Clavier mécanique | 25000
```

À retenir

• Ordre logique d'exécution : FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.

• Un alias défini dans SELECT n'est pas utilisable dans WHERE, car WHERE s'exécute avant.

• Les fonctions d'agrégation (SUM, AVG…) sont interdites dans WHERE : c'est HAVING qui filtre après calcul.

Exercice

Sur la table clients(id_client, nom, ville, email) de la boutique, écris la requête qui affiche le nom et la ville des clients habitant à Bamako, classés par ordre alphabétique de nom. Indique ensuite, dans l'ordre, les clauses que le moteur exécute.

✔ Corrigé
SELECT nom, ville FROM clients WHERE ville = 'Bamako' ORDER BY nom ; — Résultat vérifié sous SQLite (3 lignes) : Aminata Coulibaly | Bamako ; Awa Diallo | Bamako ; Moussa Traoré | Bamako. Ordre d'exécution logique : FROM clients (le moteur prend la table), WHERE ville = 'Bamako' (il ne garde que 3 lignes sur 7), SELECT nom, ville (il ne conserve que ces deux colonnes), ORDER BY nom (il trie : Aminata avant Awa avant Moussa). Remarque que le tri arrive en dernier : trier avant de filtrer ne changerait rien au résultat, mais coûterait plus cher sur une grosse table.

Leçon 2 — Filtrage avancé : WHERE, LIKE, BETWEEN et IN

Le WHERE de L1 se limitait souvent à une égalité ou une comparaison. SQL offre des opérateurs de filtrage bien plus expressifs. BETWEEN a AND b retient les valeurs dans l'intervalle [a, b], bornes COMPRISES : prix BETWEEN 10000 AND 50000 garde aussi

bien un produit à 10 000 qu'un produit à 50 000 FCFA. C'est l'équivalent raccourci de prix >= 10000 AND prix <= 50000, et cette inclusivité des bornes est une source classique d'erreurs aux examens : elle doit être connue, pas devinée.

L'opérateur IN teste l'appartenance à une liste : categorie IN ('Informatique', 'Audio') remplace avantageusement une chaîne de OR et reste lisible même avec dix valeurs. Son contraire, NOT IN, exclut la liste. Attention à un piège redoutable : si la liste (ou la sous-requête) contient une valeur NULL, NOT IN peut rendre un résultat vide et inattendu — nous y reviendrons avec les sous-requêtes.

LIKE sert à chercher dans le texte grâce à deux jokers : % remplace n'importe quelle suite de caractères (même vide) et _ remplace exactement un caractère. nom LIKE 'S%' signifie « commence par S », '%Diallo%' signifie « contient Diallo », '%X12' signifie « se termine par X12 ». Dans la plupart des moteurs, LIKE est insensible à la casse pour les lettres ASCII, mais pas toujours pour les lettres accentuées : prudence avec les É et les À. Combiner ces opérateurs avec AND, OR, NOT et des parenthèses permet d'exprimer n'importe quelle condition métier, à condition de parenthéser généreusement : AND est prioritaire sur OR, et un filtre mal parenthésé donne un résultat faux en silence.

L'essentiel
BETWEEN inclut ses deux bornes ; IN teste l'appartenance à une liste et remplace les longues chaînes de OR ; LIKE cherche dans le texte avec % (suite quelconque) et _ (un seul caractère). Les parenthèses lèvent toute ambiguïté entre AND et OR.

Exemple — Combiner IN, BETWEEN et AND sur les produits

```
-- Question : les produits Informatique ou Audio
-- coûtant entre 10 000 et 50 000 FCFA, du moins
-- cher au plus cher.
SELECT nom_produit, prix
FROM produits
WHERE categorie IN ('Informatique', 'Audio')
AND prix BETWEEN 10000 AND 50000
ORDER BY prix;

-- Résultat (3 lignes) :
-- Écouteurs Bluetooth | 12000
-- Clavier mécanique | 25000
-- Disque dur externe 1To | 45000

-- Autre exemple : LIKE 'S%' trouve 2 produits
-- (Souris sans fil, Smartphone X12).
```

À retenir

- BETWEEN est inclusif : BETWEEN 10 AND 50 garde 10 et 50.
- IN (liste) évite les OR en cascade ; NOT IN est piégeux si la liste contient NULL.
- Avec LIKE : % = zéro, un ou plusieurs caractères ; _ = exactement un caractère.

AND est prioritaire sur OR : parenthèse tes conditions.

Exercice

Sur la base boutique, écris la requête qui affiche le nom et la ville des clients dont le nom contient « Diallo » OU qui habitent à Sikasso ou à Kayes, classés par nom. Combien de lignes obtiens-tu et lesquelles ?

✔ Corrigé
SELECT nom, ville FROM clients WHERE nom LIKE '%Diallo%' OR ville IN ('Sikasso', 'Kayes') ORDER BY nom ; — Résultat vérifié sous SQLite (3 lignes) : Awa Diallo | Bamako (retenue grâce au LIKE, bien qu'elle n'habite ni Sikasso ni Kayes) ; Fatoumata Keïta | Sikasso ; Sekou Diarra | Kayes. Le OR est ici indispensable : un AND exigerait à la fois le nom Diallo ET l'une des deux villes, et ne rendrait aucune ligne. Cet exercice montre aussi que LIKE '%Diallo%' cherche la chaîne n'importe où dans le nom, pas seulement au début.

Leçon 3 — Jointures internes et externes

Dans une base relationnelle bien conçue, l'information est éclatée : la table commandes sait QUI a commandé grâce à id_client, mais le nom du client vit dans la table clients. La jointure recompose l'information en reliant les lignes de deux tables sur une condition, presque toujours l'égalité entre une clé étrangère et la clé primaire correspondante : commandes.id_client = clients.id_client. Sans jointures, une base relationnelle serait une collection de tableaux isolés et inexploitables.

La jointure interne, INNER JOIN (ou simplement JOIN), ne conserve que les lignes qui ont une correspondance des DEUX côtés. Joindre commandes et clients donne les 8 commandes avec le nom de chaque client ; en revanche, une cliente sans aucune commande disparaît du résultat, et c'est parfois très bien ainsi. La syntaxe moderne explicite la condition avec ON ; l'ancienne syntaxe (FROM a, b WHERE a.id = b.id) existe encore dans de vieux codes, mais la forme avec JOIN … ON est plus lisible et sépare clairement le lien (ON) du filtre métier (WHERE).

La jointure externe gauche, LEFT JOIN, conserve TOUTES les lignes de la table de gauche, même sans correspondance : les colonnes de la table de droite valent alors NULL. C'est l'outil des questions « qui manque à l'appel ? » : clients sans commande (avec COUNT qui vaut 0, car COUNT(colonne) ignore les NULL alors que COUNT(*) compterait la ligne), produits jamais vendus (en testant WHERE clé IS NULL), commandes sans lignes. Choisir entre JOIN et LEFT JOIN, c'est décider si les absences font partie de la réponse — une décision métier avant d'être une décision technique.

L'essentiel
La jointure relie deux tables par clé étrangère = clé primaire. INNER JOIN ne garde que les correspondances des deux côtés ; LEFT JOIN garde toute la table de gauche et remplit le reste avec des NULL — l'outil pour repérer ce qui manque.

Exemple — Détail chiffré d'une commande : jointure à trois tables

```
-- Base : commandes(id_commande, id_client,
-- date_commande, statut)
-- lignes_commande(id_commande, id_produit,
-- quantite, prix_unitaire)
-- Question : le détail de la commande 106.
SELECT p.nom_produit, lc.quantite, lc.prix_unitaire,
lc.quantite * lc.prix_unitaire AS montant
FROM lignes_commande lc
JOIN produits p ON p.id_produit = lc.id_produit
WHERE lc.id_commande = 106;

-- Résultat (2 lignes) :
-- Ordinateur portable | 1 | 350000 | 350000
-- Souris sans fil | 2 | 7500 | 15000
-- Total de la commande 106 : 365 000 FCFA.
-- La jointure commandes x clients donne, elle,
-- les 8 commandes avec le nom de leur client.
```

À retenir

• Condition de jointure = clé étrangère d'un côté, clé primaire de l'autre, reliées par ON.

• INNER JOIN fait disparaître les lignes sans correspondance ; LEFT JOIN les conserve avec des NULL.

• COUNT(colonne) ignore les NULL, COUNT(*) compte les lignes : avec un LEFT JOIN, ce choix change le résultat.

Exercice

1) Écris la requête qui donne, pour CHAQUE client de la boutique (même sans commande), son nom et son nombre de commandes. 2) Écris celle qui trouve le ou les produits jamais commandés. Donne les résultats.

✔ Corrigé

1) SELECT cl.nom, COUNT(c.id_commande) AS nb FROM clients cl LEFT JOIN commandes c ON c.id_client = cl.id_client GROUP BY cl.nom ORDER BY cl.nom ; — Résultat vérifié (7 lignes) : Aminata Coulibaly 1 ; Awa Diallo 2 ; Fatoumata Keïta 1 ; Ibrahim Sangaré 1 ; Mariam Koné 0 ; Moussa Traoré 2 ; Sekou Diarra 1. Mariam Koné apparaît avec 0 grâce au LEFT JOIN et au COUNT(c.id_commande) qui ignore le NULL ; un INNER JOIN l'aurait fait disparaître, et un COUNT(*) lui aurait faussement attribué 1 commande. 2) SELECT p.nom_produit FROM produits p LEFT JOIN lignes_commande lc ON lc.id_produit = p.id_produit WHERE lc.id_produit IS NULL ; — Résultat : une seule ligne, « Imprimante laser », le seul produit des 9 qui n'apparaît dans aucune ligne de commande. Le test IS NULL sur la clé de la table de droite est LA technique standard pour trouver les absents.

Leçon 4 — Fonctions d'agrégation et regroupements GROUP BY et HAVING

Les fonctions d'agrégation condensent plusieurs lignes en une valeur : COUNT compte, SUM additionne, AVG calcule une moyenne, MIN et MAX donnent les extrêmes. Sans GROUP BY, elles portent sur toute la table : SUM sur les lignes de commande donne le chiffre d'affaires total de la boutique, 1 171 000 FCFA toutes commandes confondues (résultat vérifié). Deux détails qui comptent : COUNT(*) compte les lignes, COUNT(colonne) ignore les lignes où la colonne est NULL ; et AVG ignore elle aussi les NULL au lieu de les traiter comme des zéros.

GROUP BY change tout : il découpe les lignes en groupes de même valeur et calcule les agrégats PAR groupe. GROUP BY categorie donne un prix moyen par catégorie ; GROUP BY statut donne un nombre de commandes par statut. La règle d'or : dans le SELECT d'une requête groupée, on ne peut afficher que les colonnes du GROUP BY et des fonctions d'agrégation — demander « le nom d'un produit » avec un GROUP BY categorie n'a pas de sens, car un groupe contient plusieurs produits. SQLite tolère ce genre d'écriture en rendant une valeur arbitraire : ne te laisse pas abuser, les autres moteurs la refusent, et elle est fausse dans l'esprit.

HAVING filtre les groupes APRÈS calcul, là où WHERE filtre les lignes AVANT. La distinction est mécanique : WHERE ne connaît pas encore les agrégats (l'ordre d'exécution de la leçon 1), donc « les clients dont le total dépasse 100 000 » s'écrit avec HAVING SUM(...) > 100000, jamais avec WHERE. Dans une requête complète, les deux cohabitent : WHERE élimine d'abord les lignes indésirables (par exemple les commandes annulées), GROUP BY forme les groupes, HAVING ne conserve que les groupes qui satisfont la condition sur l'agrégat.

L'essentiel
Les agrégats (COUNT, SUM, AVG, MIN, MAX) résument des lignes ; GROUP BY les calcule par groupe ; HAVING filtre sur le résultat des agrégats, WHERE filtre les lignes avant. Dans un SELECT groupé : uniquement les colonnes du GROUP BY et des agrégats.

Exemple — Statistiques du catalogue par catégorie

```
-- Question : combien de produits par catégorie
-- et à quel prix moyen (arrondi à 2 décimales) ?
SELECT categorie, COUNT(*) AS nb_produits,
ROUND(AVG(prix), 2) AS prix_moyen
FROM produits
GROUP BY categorie
ORDER BY categorie;

-- Résultat (4 lignes) :
-- Accessoires | 1 | 5000
-- Audio | 1 | 12000
-- Informatique| 6 | 107083.33
-- Téléphonie | 1 | 85000
```

À retenir

- COUNT(*) compte les lignes ; COUNT(colonne) et AVG ignorent les NULL.
- Avec GROUP BY, le SELECT n'affiche que les clés de groupe et des agrégats.

• WHERE filtre les lignes avant regroupement, HAVING filtre les groupes après calcul : « total supérieur à… » = HAVING.

Exercice

1) Écris la requête qui compte les commandes par statut, de la plus fréquente à la plus rare. 2) Écris celle qui donne, pour chaque client, le total de ses achats hors commandes annulées, en ne gardant que les clients dont le total dépasse 100 000 FCFA, classés par total décroissant. Donne les résultats.

✔ Corrigé
1) SELECT statut, COUNT(*) AS nb FROM commandes GROUP BY statut ORDER BY nb DESC ; — Résultat vérifié (3 lignes) : Livrée 5 ; En cours 2 ; Annulée 1. 2) SELECT cl.nom, SUM(lc.quantite * lc.prix_unitaire) AS total FROM clients cl JOIN commandes c ON c.id_client = cl.id_client JOIN lignes_commande lc ON lc.id_commande = c.id_commande WHERE c.statut <> 'Annulée' GROUP BY cl.nom HAVING total > 100000 ORDER BY total DESC ; — Résultat vérifié (3 lignes) : Awa Diallo 382 500 ; Aminata Coulibaly 365 000 ; Moussa Traoré 305 000. Observe la répartition des rôles : WHERE écarte la commande annulée 105 AVANT tout calcul (le total d'Ibrahim Sangaré devient nul et il disparaît dès la jointure interne), HAVING écarte ensuite les groupes trop petits (Fatoumata Keïta 29 000, Sekou Diarra 44 500).

Leçon 5 — Sous-requêtes et requêtes imbriquées

Une sous-requête est une requête placée à l'intérieur d'une autre, entre parenthèses. Elle sert quand la réponse dépend d'un résultat intermédiaire que l'on ne connaît pas d'avance : « les produits plus chers que le prix MOYEN » exige d'abord de calculer cette moyenne. Le moteur exécute la sous-requête, puis injecte son résultat dans la requête principale. Plutôt que de recopier une valeur lue à l'écran (82 722,22 aujourd'hui, fausse demain), la sous-requête recalcule la référence à chaque exécution : c'est la différence entre une requête qui marche un jour et une requête qui marche toujours.

Selon ce qu'elle rend, une sous-requête s'emploie différemment. Une sous-requête scalaire rend UNE valeur : elle se compare avec =, >, <, comme dans WHERE prix = (SELECT MAX(prix) FROM produits). Une sous-requête qui rend une liste de valeurs s'emploie avec IN : WHERE id_client IN (SELECT id_client FROM commandes WHERE …). Elle peut aussi apparaître dans le FROM, comme une table temporaire que l'on interroge : c'est la technique pour agréger des agrégats, par exemple moyenner les montants de commandes préalablement calculés par un GROUP BY interne.

Restent deux pièges à connaître. Premier piège : NOT IN face aux NULL — si la sous-requête rend une liste contenant un NULL, la comparaison échoue silencieusement et le résultat peut devenir vide ; on lui préfère souvent NOT EXISTS ou un LEFT JOIN avec IS NULL. Deuxième piège : les sous-requêtes corrélées, réévaluées pour chaque ligne de la table principale, sont puissantes mais lentes sur les grosses tables ; dans la majorité des cas, une jointure fait le même travail plus vite. La règle pratique : sous-requête pour la clarté quand la question est « en deux temps », jointure quand les données sont simplement éclatées entre tables.

L'essentiel
Une sous-requête calcule un résultat intermédiaire réutilisé par la requête principale : scalaire (comparaison directe), liste (avec IN), ou table dérivée dans le FROM. Méfiance : NOT IN et les NULL, et les sous-requêtes corrélées coûteuses.

Exemple — Les produits au-dessus du prix moyen (sous-requête scalaire)

```
-- Le prix moyen des 9 produits vaut 82 722,22
-- (vérifié : SELECT ROUND(AVG(prix),2) FROM produits)
-- Question : les produits plus chers que cette
-- moyenne, sans écrire la moyenne en dur.
SELECT nom_produit, prix
FROM produits
WHERE prix > (SELECT AVG(prix) FROM produits)
ORDER BY prix DESC;

-- Résultat (4 lignes) :
-- Ordinateur portable | 350000
-- Tablette 10 pouces | 120000
-- Imprimante laser | 95000
-- Smartphone X12 | 85000
```

À retenir

- Sous-requête scalaire = une valeur, comparée avec =, >, < ; liste de valeurs = IN.
- Une sous-requête dans le FROM agit comme une table temporaire (agréger des agrégats).
- NOT IN + NULL = résultat faussé en silence ; préfère NOT EXISTS ou LEFT JOIN … IS NULL.

Exercice

1) Écris, avec une sous-requête scalaire, la requête qui affiche le produit le plus cher de la boutique. 2) Écris, avec des sous-requêtes IN imbriquées, celle qui donne le nom des clients ayant commandé un ordinateur portable (produit n° 1). 3) Question avancée : quelles commandes ont un montant total supérieur à la moyenne des montants de toutes les commandes ? Donne les résultats.

✔ Corrigé
1) SELECT nom_produit, prix FROM produits WHERE prix = (SELECT MAX(prix) FROM produits) ; — Résultat : Ordinateur portable | 350 000. Le = et non le > : on cherche l'égalité avec le maximum calculé. 2) SELECT nom FROM clients WHERE id_client IN (SELECT id_client FROM commandes WHERE id_commande IN (SELECT id_commande FROM lignes_commande WHERE id_produit = 1)) ; — Résultat vérifié (2 lignes) : Awa Diallo et Aminata Coulibaly. On remonte la chaîne en trois temps : les lignes contenant le produit 1 (commandes 101 et 106), les commandes correspondantes et leurs clients, puis les noms. 3) SELECT id_commande, SUM(quantite * prix_unitaire) AS montant FROM lignes_commande GROUP BY id_commande HAVING montant > (SELECT AVG(m) FROM (SELECT SUM(quantite * prix_unitaire) AS m FROM lignes_commande GROUP BY id_commande)) ORDER BY montant DESC ; — La moyenne des montants vaut 146 375 FCFA (vérifié) et le résultat comporte 3 lignes : commande 106 (365 000), commande 101 (357 500), commande 102 (185 000). C'est le modèle « agréger des agrégats » : la sous-requête du FROM calcule d'abord un montant par commande, puis AVG en fait la moyenne.

Leçon 6 — Création et modification de tables : le DDL

Le DDL (Data Definition Language) crée et fait évoluer la structure de la base : tables, colonnes, contraintes. CREATE TABLE déclare chaque colonne avec son type (INTEGER, TEXT, REAL, DATE selon les moteurs) et ses contraintes : PRIMARY KEY identifie chaque ligne de façon unique, NOT NULL interdit les valeurs absentes, UNIQUE interdit les doublons, DEFAULT fournit une valeur par défaut, CHECK impose une condition de validité, et REFERENCES déclare une clé étrangère vers une autre table. Ces contraintes ne sont pas de la bureaucratie : elles font de la base elle-même une gardienne des données, qui refusera une note de 9 sur 5 ou une commande rattachée à un client inexistant, quel que soit le programme qui écrit.

Une base vit : ALTER TABLE fait évoluer le schéma sans tout reconstruire — ajouter une colonne (ADD COLUMN), renommer une table ou une colonne (selon les moteurs), parfois modifier un type. Les colonnes ajoutées valent NULL pour les lignes existantes : après avoir ajouté telephone à la table clients, il faut un UPDATE pour remplir celles que l'on connaît. Les opérations destructrices existent aussi : DROP TABLE supprime table et données, DROP COLUMN supprime une colonne ; elles sont irréversibles et s'emploient avec des sauvegardes et beaucoup de sang-froid.

Enfin, le DDL ne remplace ni la réflexion ni les autres familles de SQL : le DML (INSERT, UPDATE, DELETE) remplit et corrige les données dans la structure créée, et certaines différences de dialecte existent (AUTO_INCREMENT chez MySQL, SERIAL ou IDENTITY ailleurs, AUTOINCREMENT sous SQLite). La compétence professionnelle consiste à écrire des schémas où les contraintes portent les règles métier stables, puis à documenter chaque évolution : qui a ajouté quelle colonne, quand et pourquoi.

L'essentiel
Le DDL définit la structure : CREATE TABLE avec types et contraintes (PRIMARY KEY, NOT NULL, UNIQUE, CHECK, REFERENCES), ALTER TABLE pour faire évoluer, DROP pour supprimer — avec prudence. Bien placées, les contraintes protègent les données à la source.

Exemple — Créer une table avis avec contraintes, puis faire évoluer clients

```
-- Nouvelle table : les avis clients sur les
-- produits, note obligatoire entre 1 et 5.
CREATE TABLE avis (
id_avis INTEGER PRIMARY KEY,
id_produit INTEGER NOT NULL
REFERENCES produits(id_produit),
note INTEGER CHECK (note BETWEEN 1 AND 5),
commentaire TEXT
);
INSERT INTO avis VALUES
(1, 1, 5, 'Très satisfait'),
(2, 4, 4, 'Bon téléphone');
-- INSERT d'une note de 9 : REFUSÉ par le CHECK
-- (« CHECK constraint failed »), vérifié sous
-- SQLite : la contrainte protège la base.

-- Évolution : ajouter un téléphone aux clients.
ALTER TABLE clients ADD COLUMN telephone TEXT;
-- Les 7 clients existants ont telephone = NULL
-- jusqu'à un UPDATE de remplissage.
```

À retenir

- PRIMARY KEY, NOT NULL, UNIQUE, CHECK, REFERENCES : les contraintes font respecter les règles métier par la base elle-même.
- ALTER TABLE ADD COLUMN laisse les anciennes lignes à NULL : prévoir l'UPDATE de remplissage.
- DROP TABLE / DROP COLUMN sont irréversibles : sauvegarde d'abord, toujours.

Exercice

La boutique veut suivre ses fournisseurs. Écris le CREATE TABLE d'une table fournisseurs (identifiant, nom obligatoire, ville, téléphone unique) puis l'instruction pour relier chaque produit à son fournisseur. Explique le rôle de chaque contrainte choisie.

✔ Corrigé
CREATE TABLE fournisseurs (id_fournisseur INTEGER PRIMARY KEY, nom TEXT NOT NULL, ville TEXT, telephone TEXT UNIQUE) ; puis ALTER TABLE produits ADD COLUMN id_fournisseur INTEGER REFERENCES fournisseurs(id_fournisseur) ; — Rôles : PRIMARY KEY donne à chaque fournisseur une identité stable, utilisée par les clés étrangères ; NOT NULL sur nom refuse un fournisseur anonyme, inutilisable en pratique ; UNIQUE sur telephone empêche de créer deux fiches pour le même numéro (doublon classique des bases réelles) ; REFERENCES sur produits.id_fournisseur garantit qu'un produit ne peut pointer que vers un fournisseur existant — l'intégrité référentielle. Note la cohérence avec la leçon 3 : sans cette colonne et cette contrainte, aucune jointure produits × fournisseurs n'est possible proprement.

Leçon 7 — Vues, index et optimisation simple des requêtes

Une vue est une requête SELECT enregistrée sous un nom et utilisable ensuite comme une table : CREATE VIEW vue_totaux_commandes AS SELECT … . Elle ne stocke pas de données (à quelques exceptions près selon les moteurs) : chaque interrogation de la vue réexécute la requête enregistrée sur les données à jour. Les vues servent à trois choses : simplifier (les utilisateurs interrogent une « table » propre au lieu de réécrire quatre jointures), stabiliser (si le schéma évolue, on corrige la vue et les rapports continuent de marcher) et protéger (une vue peut ne montrer que certaines colonnes et lignes, masquant par exemple les données sensibles).

Un index est une structure auxiliaire — comparable à l'index d'un livre — qui permet au moteur de retrouver des lignes sans parcourir toute la table. Sans index, une recherche par id_client oblige SQLite à un SCAN complet de commandes ; après CREATE INDEX idx_commandes_client ON commandes(id_client), le plan d'exécution affiche SEARCH … USING INDEX : le moteur descend directement dans l'index. Le bénéfice croît avec la taille des tables ; sur nos 8 commandes, il est invisible à l'œil, mais sur un million de lignes, c'est le jour et la nuit.

Les index ont un prix : ils occupent de l'espace disque et ralentissent légèrement les INSERT, UPDATE et DELETE, qui doivent les mettre à jour. La démarche d'optimisation professionnelle est donc mesurée : d'abord lire le plan (EXPLAIN QUERY PLAN sous SQLite, EXPLAIN sous MySQL/PostgreSQL), indexer les colonnes souvent filtrées ou jointes (clés étrangères en tête), éviter SELECT * quand trois colonnes suffisent, et ne jamais indexer « au cas où ». Optimiser, ce n'est pas ajouter des index partout : c'est comprendre pourquoi CETTE requête est lente et corriger précisément cette cause.

L'essentiel
Une vue = une requête nommée, interrogée comme une table, toujours à jour, qui simplifie et protège. Un index = un accès direct aux lignes qui transforme un SCAN en SEARCH, au prix d'un peu d'espace et d'écritures plus lentes. On optimise en lisant le plan d'exécution (EXPLAIN), pas au hasard.

Exemple — Une vue des totaux de commandes et un index vérifié par EXPLAIN

```
CREATE VIEW vue_totaux_commandes AS
SELECT c.id_commande, cl.nom AS client, c.statut,
SUM(lc.quantite * lc.prix_unitaire) AS total
FROM commandes c
JOIN clients cl ON cl.id_client = c.id_client
JOIN lignes_commande lc
ON lc.id_commande = c.id_commande
GROUP BY c.id_commande, cl.nom, c.statut;

-- Utilisation (3 plus grosses commandes) :
SELECT * FROM vue_totaux_commandes
ORDER BY total DESC LIMIT 3;
-- 106 | Aminata Coulibaly | Livrée | 365000
-- 101 | Awa Diallo | Livrée | 357500
-- 102 | Moussa Traoré | Livrée | 185000

-- Index : avant, EXPLAIN QUERY PLAN affichait
-- « SCAN commandes » ; après :
CREATE INDEX idx_commandes_client
ON commandes(id_client);
-- « SEARCH commandes USING INDEX
-- idx_commandes_client (id_client=?) »
```

À retenir

• CREATE VIEW nom AS SELECT … : la vue ne stocke rien, elle réexécute sa requête — données toujours fraîches.

• CREATE INDEX accélère les recherches et jointures sur la colonne indexée, mais coûte de l'espace et ralentit les écritures.

• EXPLAIN (QUERY PLAN) avant/après : SCAN = parcours complet, SEARCH USING INDEX = accès direct. On indexe les clés étrangères et les colonnes de filtre fréquentes.

Exercice

1) Crée une vue vue_produits_chers qui expose le nom, la catégorie et le prix des produits de plus de 50 000 FCFA, puis interroge-la pour la seule catégorie Informatique : que rends-tu ? 2) Un collègue propose d'indexer toutes les colonnes de toutes les tables « pour que tout aille plus vite » : donne deux arguments contre.

✔ Corrigé
1) CREATE VIEW vue_produits_chers AS SELECT nom_produit, categorie, prix FROM produits WHERE prix > 50000 ; puis SELECT * FROM vue_produits_chers WHERE categorie = 'Informatique' ; — Résultat obtenu en appliquant la vue à notre base (vérifié par la requête équivalente) : Ordinateur portable 350 000 ; Tablette 10 pouces 120 000 ; Imprimante laser 95 000, soit 3 lignes. Remarque la mécanique : la vue applique déjà son filtre (prix > 50 000, ce qui écarte aussi le Smartphone X12 de Téléphonie à 85 000… non : 85 000 > 50 000, il est DANS la vue, mais le WHERE categorie = 'Informatique' de la requête l'écarte du résultat final). Les filtres se cumulent, la vue n'est pas une copie figée. 2) Contre l'indexation totale : (a) chaque index doit être mis à jour à chaque INSERT/UPDATE/DELETE, donc les écritures ralentissent et la base grossit ; (b) un index sur une colonne peu sélective (statut avec 3 valeurs, par exemple) n'aide presque jamais le moteur, qui préfère souvent un SCAN. On indexe donc de manière ciblée, après lecture du plan d'exécution.

Travaux dirigés (TD)

TD 1 — Rapport d'activité mensuel de la boutique

La gérante de la boutique veut un rapport fiable avant de commander de nouveaux stocks. En utilisant uniquement la base d'exemple du cours (commandes et lignes, en EXCLUANT partout les commandes annulées), écris les requêtes qui donnent : 1) le chiffre d'affaires par catégorie de produits, de la plus rentable à la moins rentable ; 2) le chiffre d'affaires total correspondant ; 3) la liste des clientes et clients avec le nombre de commandes (hors annulées) et le total acheté, pour remercier les plus fidèles. Donne les résultats numériques obtenus.

✔ Corrigé du TD 1
1) SELECT p.categorie, SUM(lc.quantite * lc.prix_unitaire) AS ca FROM lignes_commande lc JOIN produits p ON p.id_produit = lc.id_produit JOIN commandes c ON c.id_commande = lc.id_commande WHERE c.statut <> 'Annulée' GROUP BY p.categorie ORDER BY ca DESC ; — Résultat vérifié sous SQLite (4 lignes) : Informatique 900 000 ; Téléphonie 170 000 ; Audio 36 000 ; Accessoires 20 000. La troisième jointure vers commandes est indispensable : c'est elle qui donne accès au statut pour exclure la commande 105. 2) La même somme sans GROUP BY donne 1 126 000 FCFA (vérifié) — cohérent avec la somme des quatre catégories : 900 000 + 170 000 + 36 000 + 20 000 = 1 126 000. Ce contrôle croisé est un réflexe professionnel : un rapport dont les parties ne se totalisent pas est un rapport faux. 3) SELECT cl.nom, COUNT(DISTINCT c.id_commande) AS nb, SUM(lc.quantite * lc.prix_unitaire) AS total FROM clients cl JOIN commandes c ON c.id_client = cl.id_client JOIN lignes_commande lc ON lc.id_commande = c.id_commande WHERE c.statut <> 'Annulée' GROUP BY cl.nom ORDER BY total DESC ; — Résultats : Awa Diallo 2 commandes, 382 500 ; Aminata Coulibaly 1 commande, 365 000 ; Moussa Traoré 2 commandes, 305 000 ; Sekou Diarra 1 commande, 44 500 ; Fatoumata Keïta 1 commande, 29 000. Attention au COUNT(DISTINCT c.id_commande) : compter les lignes de lignes_commande (COUNT(*)) aurait donné le nombre de LIGNES (3 pour Awa) et non le nombre de commandes — piège classique des jointures en éventail.

TD 2 — Audit de cohérence et de complétude de la base

Avant de migrer la boutique vers un nouveau logiciel, on audite la base : il faut repérer les données orphelines ou jamais exploitées. Écris les requêtes d'audit suivantes et donne leurs résultats : 1) le nombre de lignes de détail de chaque commande, y compris pour d'éventuelles commandes vides ; 2) les clients qui n'ont jamais passé commande ; 3) les produits qui n'ont jamais été vendus ; 4) le total général de toutes les lignes de commande, à comparer avec la somme des totaux par commande de la vue vue_totaux_commandes.

✔ Corrigé du TD 2

1) SELECT c.id_commande, COUNT(lc.id_produit) AS nb_lignes FROM commandes c LEFT JOIN lignes_commande lc ON lc.id_commande = c.id_commande GROUP BY c.id_commande ORDER BY c.id_commande ; — Résultat vérifié (8 lignes) : 101 → 2 ; 102 → 2 ; 103 → 1 ; 104 → 2 ; 105 → 1 ; 106 → 2 ; 107 → 1 ; 108 → 3. Le LEFT JOIN est la clé de l'audit : avec un INNER JOIN, une commande vide aurait disparu de l'audit au lieu d'apparaître avec 0 — ici il n'y en a aucune, la base est complète sur ce point. 2) SELECT nom, ville FROM clients WHERE id_client NOT IN (SELECT id_client FROM commandes) ; — Résultat : Mariam Koné (Koulikoro), seule cliente sans commande. (La même réponse s'obtient par LEFT JOIN … IS NULL.) 3) Par LEFT JOIN sur lignes_commande avec IS NULL : un seul produit, l'Imprimante laser — candidat évident à une promotion ou à un déstockage. 4) SELECT SUM(quantite * prix_unitaire) FROM lignes_commande ; — Résultat vérifié : 1 171 000 FCFA, toutes commandes confondues (y compris l'annulée 105 et ses 45 000). La vue vue_totaux_commandes, qui inclut aussi l'annulée, totalise le même montant : l'audit passe. L'écart avec le 1 126 000 du TD 1 s'explique exactement : 1 171 000 − 1 126 000 = 45 000, le montant de la commande annulée — un audit réussi est un audit où chaque écart s'explique au franc près.

Travaux pratiques (TP)

TP 1 — Construire la base boutique de zéro et la valider

Objectif : reproduire l'environnement du cours et prouver qu'il fonctionne. Étape 1 : dans SQLite (ou DB Browser), crée les quatre tables clients, produits, commandes et lignes_commande avec leurs clés primaires et étrangères, en t'inspirant de la leçon 6. Étape 2 : insère le jeu d'essai du cours : 7 clients (dont Mariam Koné à Koulikoro), 9 produits (dont l'Imprimante laser à 95 000 FCFA), 8 commandes de statuts variés et 14 lignes de commande. Étape 3 : écris un script de validation qui exécute et affiche : le nombre de lignes de chaque table, le prix moyen des produits, le total des ventes, la plus grosse commande. Étape 4 : crée la vue vue_totaux_commandes et l'index idx_commandes_client, puis compare le plan d'exécution d'une recherche par client avant et après l'index.

✔ Attendus du TP 1
Résultats attendus, tous vérifiés sous SQLite : effectifs — clients 7, produits 9, commandes 8, lignes_commande 14. Prix moyen des produits : 82 722,22 FCFA. Total des ventes (somme de quantite × prix_unitaire sur toutes les lignes) : 1 171 000 FCFA. Plus grosse commande d'après la vue : la n° 106 (Aminata Coulibaly, 365 000 FCFA), devant la 101 (357 500) et la 102 (185 000). Plan d'exécution de SELECT * FROM commandes WHERE id_client = 1 : « SCAN commandes » avant l'index, « SEARCH commandes USING INDEX idx_commandes_client (id_client=?) » après. Si un seul de tes chiffres diffère, ton jeu d'essai diffère du cours : compare ligne à ligne, c'est l'objet même de ce TP — une base de test n'a de valeur que si ses résultats de référence sont connus et re-vérifiés après chaque modification.

Exercices corrigés supplémentaires

Exercice 1

Écris la requête qui donne les 3 produits les plus vendus en quantité totale (toutes commandes confondues), avec la quantité vendue pour chacun. En cas d'égalité, départage par ordre alphabétique du nom.

✔ Corrigé
SELECT p.nom_produit, SUM(lc.quantite) AS qte_totale FROM produits p JOIN lignes_commande lc ON lc.id_produit = p.id_produit GROUP BY p.nom_produit ORDER BY qte_totale DESC, p.nom_produit LIMIT 3 ; — Résultat vérifié sous SQLite (3 lignes) : Chargeur USB-C 4 ; Souris sans fil 4 ; Écouteurs Bluetooth 3. Deux produits sont à égalité à 4 unités : sans le second critère de tri (le nom), l'ordre entre le Chargeur et la Souris serait imprévisible d'un moteur à l'autre — d'où l'importance de départager explicitement dans les « top N ».

Exercice 2

La boutique prépare une solde : elle veut afficher, pour les produits Informatique dont le stock est inférieur à 30, le prix actuel et le prix après 10 % de remise (arrondi). Écris la requête et donne son résultat.

✔ Corrigé
SELECT nom_produit, prix, ROUND(prix * 0.9) AS prix_remise FROM produits WHERE categorie = 'Informatique' AND stock < 30 ORDER BY prix ; — Résultat vérifié (4 lignes) : Disque dur externe 1To 45 000 → 40 500 ; Imprimante laser 95 000 → 85 500 ; Tablette 10 pouces 120 000 → 108 000 ; Ordinateur portable 350 000 → 315 000. La Souris sans fil (stock 150) et le Clavier mécanique (stock 40) sont écartés par le filtre de stock. Remarque : ROUND évite les prix à virgule ; en production, on déciderait aussi si la remise s'applique dans un UPDATE (prix définitif) ou seulement à l'affichage — ici, c'est un calcul d'affichage, la base n'est pas modifiée.

Exercice 3

Sans utiliser de jointure, écris une requête qui affiche les clients (nom et ville) ayant passé au moins une commande « En cours », en utilisant EXISTS ou IN. Donne le résultat, puis dis en une phrase quelle écriture tu choisirais en pratique et pourquoi.

✔ Corrigé
SELECT nom, ville FROM clients WHERE id_client IN (SELECT id_client FROM commandes WHERE statut = 'En cours') ; — Résultat vérifié par le jeu d'essai : Moussa Traoré (Bamako) pour la commande 107 et Awa Diallo (Bamako) pour la commande 103, soit 2 lignes. En pratique, une jointure JOIN commandes avec un DISTINCT donnerait exactement la même réponse et reste souvent plus lisible quand on doit aussi afficher des colonnes de commandes ; la sous-requête IN brille quand on ne veut QUE des colonnes de la table principale, comme ici. EXISTS serait le choix le plus robuste si la colonne pouvait contenir des NULL — réflexe à conserver pour les bases réelles.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Dans une requête SELECT, quel est l'ordre LOGIQUE d'exécution des clauses ?

A) SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY

B) FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY

C) FROM, SELECT, HAVING, WHERE, GROUP BY, ORDER BY

2. Que vaut le résultat de : prix BETWEEN 10000 AND 50000 pour un produit à exactement 50 000 FCFA ?

A) Il est exclu, BETWEEN rejette les bornes

B) Il est inclus, BETWEEN comprend ses deux bornes

C) Erreur : BETWEEN exige des valeurs strictement différentes

3. Dans LIKE, le joker % remplace…

A) Exactement un caractère

B) Un seul chiffre

C) N'importe quelle suite de caractères, même vide

4. Quelle est la différence entre WHERE et HAVING ?

A) Aucune, ce sont des synonymes

B) WHERE filtre les lignes avant regroupement, HAVING filtre les groupes après agrégation

C) HAVING remplace JOIN pour relier les tables

5. Avec un LEFT JOIN de clients vers commandes, une cliente sans aucune commande…

A) Disparaît du résultat

B) Apparaît avec des NULL dans les colonnes de commandes

C) Provoque une erreur de clé étrangère

6. Pourquoi COUNT(c.id_commande) plutôt que COUNT(*) dans un comptage par client avec LEFT JOIN ?

A) COUNT(*) est interdit avec les jointures

B) COUNT(*) compterait 1 pour un client sans commande, COUNT(colonne) ignore les NULL et donne 0

C) Les deux donnent toujours le même résultat

7. Une sous-requête qui rend une seule valeur (le prix moyen, le maximum…) s'utilise typiquement…

A) Avec l'opérateur IN uniquement

B) Dans le FROM, jamais dans le WHERE

C) Directement comparée avec =, > ou < dans le WHERE

8. À quoi sert principalement une vue (VIEW) ?

A) À stocker une copie physique des données pour aller plus vite

B) À enregistrer une requête sous un nom, interrogable comme une table et toujours à jour

C) À remplacer les clés étrangères

9. Après CREATE INDEX sur commandes(id_client), le plan d'exécution d'une recherche par client passe typiquement de…

A) SEARCH à SCAN

B) SCAN (parcours complet) à SEARCH USING INDEX (accès direct)

C) Aucun changement : les index ne concernent que les tris

10. La contrainte CHECK (note BETWEEN 1 AND 5) sur la table avis a pour effet…

A) De trier les avis par note

B) De refuser toute insertion d'une note hors de 1 à 5, par la base elle-même

C) De mettre automatiquement la note à 5

Corrigés du QCM

1. Réponse B — Le moteur part des tables (FROM), filtre les lignes (WHERE), regroupe (GROUP BY), filtre les groupes (HAVING), puis seulement calcule le SELECT, trie (ORDER BY) et limite.

2. Réponse B — BETWEEN est inclusif des deux côtés : c'est le raccourci de prix >= 10000 AND prix <= 50000.

3. Réponse C — % accepte zéro, un ou plusieurs caractères ; c'est _ (souligné) qui remplace exactement un caractère.

4. Réponse B — WHERE s'exécute avant GROUP BY et ne peut pas utiliser les agrégats ; HAVING s'exécute après et filtre sur SUM, COUNT, etc.

5. Réponse B — Le LEFT JOIN conserve toute la table de gauche ; c'est ce qui permet de repérer les absents, puis de compter 0 avec COUNT(c.id_commande).

6. Réponse B — COUNT(colonne) ne compte que les valeurs non NULL : pour Mariam Koné, la ligne « fantôme » du LEFT JOIN vaut NULL côté commande, d'où 0, contre 1 avec COUNT(*).

7. Réponse C — C'est la sous-requête scalaire : WHERE prix > (SELECT AVG(prix) FROM produits). Les listes de valeurs relèvent de IN, les tables dérivées du FROM.

8. Réponse B — Une vue ne stocke pas les données : chaque usage réexécute la requête enregistrée. Elle simplifie, stabilise et peut masquer des colonnes sensibles.

9. Réponse B — Vérifié sous SQLite avec EXPLAIN QUERY PLAN : « SCAN commandes » avant, « SEARCH commandes USING INDEX idx_commandes_client » après.

10. Réponse B — Les contraintes du DDL sont appliquées par le moteur à chaque écriture : l'insertion d'une note de 9 est rejetée avec une erreur d'intégrité, quel que soit le programme utilisé.

Résumé — les notions importantes

• Ordre logique d'exécution à connaître par cœur : FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT ; il explique la place des alias et l'interdiction des agrégats dans WHERE.

• Filtrage fin : BETWEEN inclusif, IN pour les listes (méfiance avec NOT IN et les NULL), LIKE avec % et _ pour le texte, parenthèses pour maîtriser AND/OR.

• Jointures : INNER JOIN pour les correspondances certaines, LEFT JOIN pour conserver les absents (NULL) ; COUNT(colonne) ignore les NULL, COUNT(*) compte les lignes.

• Agrégation : COUNT, SUM, AVG, MIN, MAX calculés par groupe avec GROUP BY ; WHERE filtre les lignes avant, HAVING filtre les groupes après.

• Sous-requêtes : scalaire pour comparer à une valeur calculée, IN pour une liste, table dérivée dans le FROM pour agréger des agrégats — toujours recalculées, jamais recopiées en dur.

• DDL : CREATE TABLE et ses contraintes (PRIMARY KEY, NOT NULL, UNIQUE, CHECK, REFERENCES) protègent les données à la source ; ALTER TABLE fait évoluer le schéma, DROP détruit — sauvegarde d'abord.

• Vues et index : la vue simplifie et stabilise l'accès (requête nommée, toujours à jour) ; l'index transforme un SCAN en SEARCH au prix d'écritures plus lentes — on optimise avec EXPLAIN, pas au hasard.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Le langage SQL : structure et ordre d'exécution des requêtes
Ordre logique d'exécution : FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. ; Un alias défini dans SELECT n'est pas utilisable dans WHERE, car WHERE s'exécute avant. ; Les fonctions d'agrégation (SUM, AVG…) sont interdites dans WHERE : c'est HAVING qui filtre après calcul.

Fiche 2 — Filtrage avancé : WHERE, LIKE, BETWEEN et IN
BETWEEN est inclusif : BETWEEN 10 AND 50 garde 10 et 50. ; IN (liste) évite les OR en cascade ; NOT IN est piégeux si la liste contient NULL. ; Avec LIKE : % = zéro, un ou plusieurs caractères ; _ = exactement un caractère. AND est prioritaire sur OR : parenthèse tes conditions.

Fiche 3 — Jointures internes et externes
Condition de jointure = clé étrangère d'un côté, clé primaire de l'autre, reliées par ON. ; INNER JOIN fait disparaître les lignes sans correspondance ; LEFT JOIN les conserve avec des NULL. ; COUNT(colonne) ignore les NULL, COUNT(*) compte les lignes : avec un LEFT JOIN, ce choix change le résultat.

Fiche 4 — Fonctions d'agrégation et regroupements GROUP BY et HAVING
COUNT(*) compte les lignes ; COUNT(colonne) et AVG ignorent les NULL. ; Avec GROUP BY, le SELECT n'affiche que les clés de groupe et des agrégats. ; WHERE filtre les lignes avant regroupement, HAVING filtre les groupes après calcul : « total supérieur à… » = HAVING.

Fiche 5 — Sous-requêtes et requêtes imbriquées
Sous-requête scalaire = une valeur, comparée avec =, >, < ; liste de valeurs = IN. ; Une sous-requête dans le FROM agit comme une table temporaire (agréger des agrégats). ; NOT IN + NULL = résultat faussé en silence ; préfère NOT EXISTS ou LEFT JOIN … IS NULL.

Fiche 6 — Création et modification de tables : le DDL
PRIMARY KEY, NOT NULL, UNIQUE, CHECK, REFERENCES : les contraintes font respecter les règles métier par la base elle-même. ; ALTER TABLE ADD COLUMN laisse les anciennes lignes à NULL : prévoir l'UPDATE de remplissage. ; DROP TABLE / DROP COLUMN sont irréversibles : sauvegarde d'abord, toujours.

Fiche 7 — Vues, index et optimisation simple des requêtes
CREATE VIEW nom AS SELECT … : la vue ne stocke rien, elle réexécute sa requête — données toujours fraîches. ; CREATE INDEX accélère les recherches et jointures sur la colonne indexée, mais coûte de l'espace et ralentit les écritures. ; EXPLAIN (QUERY PLAN) avant/après : SCAN = parcours complet, SEARCH USING INDEX = accès direct. On indexe les clés étrangères et les colonnes de filtre fréquentes.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — FORMATION SQL COMPLÈTE 6H : Débutant → Avancé + Projet RH de A à Z | PostgreSQL

Chaîne : Ceemtic Analytics · Langue : Français · Niveau : Débutant à avancé

Une formation complète qui couvre dans l'ordre le programme de ce cours : filtres, fonctions d'agrégation, GROUP BY et HAVING, jointures, vues et index, avec des exercices pratiques. À regarder en plusieurs séances, en t'arrêtant pour refaire chaque requête sur la base boutique.

https://www.youtube.com/watch?v=Lp9R5PlZeRg

Leçon 2 — Apprendre SQL pour Débutant - Tutoriel SQL COMPLET

Chaîne : Dataseito · Langue : Français · Niveau : Débutant

Un tutoriel complet qui repart des fondations (SELECT, WHERE, ORDER BY) avant d'enchaîner sur GROUP BY, HAVING et toutes les jointures. Parfait pour consolider les leçons 1, 2 et 4 si certains réflexes de L1 sont lointains.

https://www.youtube.com/watch?v=eL80VI4QGTg

Leçon 3 — Utiliser INNER JOIN & LEFT OUTER JOIN dans une requête SQL

Chaîne : Antoine Moulin · Langue : Français · Niveau : Débutant à intermédiaire

Le complément direct de la leçon 3 : quand utiliser INNER JOIN, quand passer en LEFT OUTER JOIN, et ce que deviennent les lignes sans correspondance. Regarde-la juste après avoir fait l'exercice sur les clients sans commande.

https://www.youtube.com/watch?v=s5hXALXyv1I

Leçon 3 — Requêtes SQL : WHERE (jointures) - Explications & Exemples - Bases de données

Chaîne : Fabinou · Langue : Français · Niveau : Débutant (niveau Licence/BTS)

Des exemples progressifs de jointures réalisées avec WHERE, avec les questions de préfixage des colonnes quand deux tables ont des champs de même nom. Utile pour reconnaître l'ancienne syntaxe que tu croiseras dans des codes existants.

https://www.youtube.com/watch?v=ZqFco3CBMSM

Leçon 5 — Langage SQL - Vidéo 23: Les Sous-Requêtes (1/4) - Introduction

Chaîne : KLN Technology · Langue : Français · Niveau : Intermédiaire

La première vidéo d'une série dédiée aux sous-requêtes : définition, sous-requêtes internes et externes, indépendantes et corrélées, et les emplacements possibles dans une requête. Le prolongement naturel de la leçon 5, à suivre avec les parties suivantes de la série.

https://www.youtube.com/watch?v=U86eaxkXs2M

Sources et références

- Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.
- Documentation officielle SQLite (sqlite.org/lang.html) — la référence exacte pour le dialecte utilisé dans les tests de ce cours : SELECT, EXPLAIN QUERY PLAN, CREATE

VIEW, CREATE INDEX.

• Documentation MySQL 8.0, chapitre « Tutorial » et « Functions and Operators » (dev.mysql.com/doc) — pour les écarts de dialecte avec MySQL (LIMIT, types, AUTO_INCREMENT).

• C. J. Date — « SQL et théorie relationnelle » (O'Reilly) : pour comprendre en profondeur pourquoi jointures, NULL et regroupements se comportent comme ils le font.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !

Mes notes

Utilise ces pages pour recopier les définitions, refaire les calculs et noter tes questions pour le prochain cours.